# Nesne Yönelimli Programlama

## Kısa Tanım Soruları

(Türkçe)

Sekiz konunun sonundaki "What is it?" tanımları tek yerde

10 Kasım 2026

**Nasıl kullanılır?**

- Bu sayfa, her konu anlatımının sonundaki **Kısa tanım soruları** bölümünü bir araya toplar; aynı tanımlar konu sayfalarının sonunda da duruyor (iki yer aynı kaynaktan üretilir, birbirinden farklılaşmaz).
- Sınavda "What is …?", "Define …" ya da test sorusu olarak gelirler (ör. *regression fault, moving target, modern maintenance, cohesion, coupling, actor, use case, aggregation*). Bu sürümde tanımların **Türkçe** karşılıkları verildi (terimin Türkçesi parantez içinde). Türkçe yalnızca anlamak için; sınavda tanımları **İngilizce** yazın.
- Kendini sınamak için: terimi oku, tanımı kapatıp kendi cümlenle İngilizce yaz, sonra karşılaştır. Ayrıntısı için ilgili konunun sayfasına bak.

## 1 Yazılım Mühendisliğinin Kapsamı (Chapter 1)

**Software crisis** (Yazılım krizi) Yazılımın geç, bütçeyi aşarak ve içinde kalıntı hatalarla teslim edilmesi.

**Regression fault** (Regresyon hatası) Başka bir yerde yapılan değişiklik yüzünden yazılımın görünüşte ilgisiz bir bölümünde ortaya çıkan hata. Information hiding regresyon hatalarını azaltır.

**Regression testing** (Regresyon testi) Bir değişiklikten sonra başka bir şeyin bozulmadığını görmek için önceki bütün test durumlarının yeniden çalıştırılması.

**Moving target problem** (Hareketli hedef problemi) Yazılım geliştirilirken gereksinimlerin değişmesi (Chapter 2).

**Verification / validation** (Doğrulama / geçerleme) Verification: her fazın sonunda yapılan test. Validation: projenin sonunda yapılan test (ürün spesifikasyonu karşılıyor mu).

**Modern maintenance** (Modern bakım) Yazılım ne zaman değiştirilirse bakım vardır; kurulumdan önce ya da sonra olması fark etmez.

**Object** (Nesne) Hem veriyi hem de o veri üzerinde yapılan eylemleri içeren yazılım bileşeni.

**Mistake / fault / failure / error** İnsan bir yanlışlık (mistake) yapar; bunun sonucu kodda bir hatadır (fault); failure, hatanın yol açtığı gözlenen yanlış davranıştır; error, sonucun ne kadar yanlış olduğunun ölçüsüdür (IEEE 610.12).

**Information hiding** (Bilgi gizleme) Bir modülün gerçekleştirme ayrıntılarının, diğer modüller onlara bağımlı olamayacak şekilde gizlenmesi; gelecekteki değişiklikler yerel kalır.

**Encapsulation** (Kapsülleme) Gerçek dünyadaki bir varlıkla ilgili her şeyin (veri ve eylemler) tek bir birimde toplanması.

**Responsibility-driven design** (Sorumluluk güdümlü tasarım) Bir nesneye, işi nasıl yaptığını bilmeden mesaj göndermek (design by contract).

## 2 Yaşam Döngüsü Modelleri (Chapter 2)

**Moving target problem** (Hareketli hedef problemi) Yazılım ürünü geliştirilirken gereksinimlerin değişmesi. Bağımlılıklar oluşturur ve regresyon hatalarına yol açabilir. Çözümü yoktur; yalnızca etkisi azaltılabilir.

**Regression fault** (Regresyon hatası) Bir değişiklik yüzünden yazılımın görünüşte ilgisiz bir bölümünde ortaya çıkan hata.

**Miller's law** (Miller yasası) İnsan aynı anda yalnızca yaklaşık yedi bilgi parçasına odaklanabilir.

**Stepwise refinement** (Adım adım iyileştirme) O an en önemli olan konulara odaklan, daha az kritik olanları ertele; her konu sonunda önem sırasına göre ele alınır.

**Iteration** (Yineleme) Her ardışık sürüm hedefe bir öncekinden daha yakındır.

**Incrementation** (Artırım) İşlevlerin adım adım eklenmesi; ürün bir dizi artırım (mini proje) olarak geliştirilir.

**Baseline** (Temel sürüm) Belirli bir andaki artifact'ların tam kümesi (ör. bir bölümün sonunda).

**Robustness** (Sağlamlık) Dağılmadan genişlemeleri ve değişiklikleri kaldırabilme özelliği.

## 3 Unified Process (Chapter 3)

**Workflow vs phase** (İş akışı ve faz) İş akışı bir adımın teknik bağlamı, faz ise iş (ekonomik) bağlamıdır. Her adım bir iş akışına ve bir faza düşer.

**Business case** (İş gerekçesi) Ürünün kapsamını ve finansal ayrıntılarını (gelir projeksiyonları, maliyet kestirimleri ya da maliyet–fayda analizi) anlatan belge; ilk sürümünü hazırlamak inception fazının asıl amacıdır.

**Beta release** (Beta sürüm) Construction fazının sonunda üretilen ilk işletme kalitesindeki sürüm.

**Traceability** (İzlenebilirlik) Bir artifact'taki her öğe bir önceki artifact'taki bir öğeye izlenebilmelidir.

**Specification document** (Spesifikasyon belgesi) Müşteri ile geliştirici arasında bir sözleşmedir; çelişki, eksiklik ve tamamlanmamışlık içermemelidir.

**Three major risks** (Üç ana risk) Teknik riskler, gereksinimleri doğru belirleyememe riski ve mimariyi doğru kuramama riski.

## 4 Modüller, Cohesion ve Coupling (Chapter 7)

**Module** (Modül) Sınır öğeleriyle çevrili, bir toplu tanımlayıcısı (ismi) olan, kodda bitişik program komutları dizisi.

**Cohesion** (Uyum) Bir modülün içindeki etkileşimin derecesi. (Yedi seviye: coincidental, logical, temporal, procedural, communicational, functional, informational.)

**Coupling** (Bağlılık) İki modül arasındaki etkileşimin derecesi. (Beş seviye: content, common, control, stamp, data.)

**Good design** (İyi tasarım) Yüksek cohesion ve düşük coupling (modül içinde en çok, modüller arasında en az etkileşim).

**Data encapsulation** (Veri kapsülleme) Bir veri yapısı ile o veri yapısı üzerinde yapılan işlemlerin bir arada olması.

**Abstract data type** (Soyut veri tipi) Bir veri tipi ile o tipin örnekleri üzerinde yapılan işlemler (informational cohesion'a sahip modül; bir sınıf).

**Information hiding** (Bilgi gizleme) Modülleri, değişmesi muhtemel öğeler gizlenecek şekilde tasarlamak; gelecekteki değişiklik yerel kalır.

**Class** (Sınıf) Kalıtımı destekleyen soyut veri tipi.

**Dynamic binding** (Dinamik bağlama) Çağrılacak metot, nesnenin sınıfına göre çalışma zamanında belirlenir.

**Polymorphism** (Çok biçimlilik) Aynı metot farklı sınıfların nesnelerine uygulanabilir ve her biri kendine göre davranır.

**Fragile base class problem** (Kırılgan taban sınıf problemi) Taban sınıftaki bir değişiklik bütün alt sınıflarına dalga gibi yayılır.

## 5 ISO/IEC 25010 Kalite Modeli

**ISO/IEC 25010** ISO/IEC 25000 (SQuaRE) serisinin kalite modeli; bir yazılım ürünü değerlendirilirken hangi kalite karakteristiklerinin dikkate alınacağını tanımlar (2023: dokuz karakteristik, 40 alt karakteristik).

**Quality (of a system)** (Bir sistemin kalitesi) Sistemin paydaşlarının belirtilmiş ve örtük ihtiyaçlarını karşılama derecesi.

**Functional suitability** (İşlevsel uygunluk) Ürünün belirtilmiş ve örtük ihtiyaçları karşılayan işlevleri sağlama derecesi (tamlık, doğruluk, yerindelik).

**Performance efficiency** (Performans verimliliği) Kullanılan kaynak miktarına göre performans (zaman davranışı, kaynak kullanımı, kapasite).

**Compatibility** (Uyumluluk) Ürünün başka ürünlerle bilgi alışverişi yapabilme ve aynı ortamı paylaşabilme derecesi (birlikte var olabilme, birlikte çalışabilirlik).

**Interaction capability** (Etkileşim yeteneği) Ürünün belirli kullanıcılarca etkileşime girilebilme derecesi (eski adı usability: öğrenilebilirlik, çalıştırılabilirlik, kullanıcı hatasından koruma, kapsayıcılık…).

**Reliability** (Güvenilirlik) Sistemin belirtilen işlevleri belirtilen koşullarda belirtilen süre boyunca yerine getirme derecesi (hatasızlık, erişilebilirlik, hata toleransı, kurtarılabilirlik).

**Security** (Güvenlik) Ürünün bilgiyi ve veriyi yetki düzeyine uygun şekilde koruma derecesi (gizlilik, bütünlük, inkâr edilemezlik, hesap verebilirlik, kimlik doğruluğu, direnç).

**Non-repudiation** (İnkâr edilemezlik) Eylemlerin ya da olayların gerçekleştiğinin kanıtlanabilmesi, böylece sonradan inkâr edilememesi.

**Maintainability** (Bakım yapılabilirlik) Ürünün etkin ve verimli biçimde değiştirilebilme derecesi (modülerlik, yeniden kullanılabilirlik, analiz edilebilirlik, değiştirilebilirlik, test edilebilirlik).

**Flexibility** (Esneklik) Ürünün gereksinim, kullanım bağlamı ya da ortam değişikliklerine uyarlanabilme derecesi (eski adı portability: uyarlanabilirlik, ölçeklenebilirlik, kurulabilirlik, yer değiştirebilirlik).

**Safety** (Emniyet) Ürünün insan hayatını, sağlığı, mülkü ya da çevreyi tehlikeye atan durumlardan kaçınma derecesi (operasyonel kısıt, risk tanımlama, arızada güvenli durum, tehlike uyarısı, güvenli bütünleşme).

## 6 Gereksinimler ve Use Case (Chapter 11)

**Use case** (Kullanım durumu) Yazılım ürününün kendisi ile kullanıcıları (aktörler) arasındaki bir etkileşimi modeller.

**Actor** (Aktör) Yazılım ürününün dışındaki dünyanın bir üyesi; bir rol (kullanıcı, başlatıcı ya da kritik rol oynayan biri). İnsan olmak zorunda değildir.

**Business model** (İş modeli) Bir kurumun iş süreçlerinin tanımı.

**Glossary** (Sözlük) Alanda kullanılan teknik kelimelerin ve anlamlarının listesi.

**Structured / unstructured interview** (Yapılandırılmış / yapılandırılmamış görüşme) Önceden planlanmış, çoğunlukla kapalı uçlu sorular / alınan cevaplara göre sorulan, çoğunlukla açık uçlu sorular.

**«include»** Yeniden kullanım için bir use case'ten dışarı alınmış davranış; ana use case bunu her zaman yapar. Ok ana use case'ten içerilen use case'e doğrudur.

**«extend»** Ana akıştan dışarı alınmış istisnai ya da nadir çalışan durumlar. Ok genişleten use case'ten genişletilen (ana) use case'e doğrudur.

**Functional / nonfunctional requirement** (İşlevsel / işlevsel olmayan gereksinim) Ürünün yapması gereken bir eylem / ürünün kendisinin bir özelliği (platform, yanıt süresi, güvenilirlik).

## 7 Nesne Yönelimli Analiz (Chapter 13)

**Entity / boundary / control class** (Varlık / sınır / kontrol sınıfı) Uzun ömürlü bilgiyi modeller / ürün ile çevresi arasındaki etkileşimi modeller (giriş, çıkış, raporlar) / karmaşık hesaplama ve algoritmaları modeller.

**Scenario** (Senaryo) Bir use case'in bir örneği.

**Noun extraction** (İsim çıkarma) Ürünü tek paragrafta tanımla, isimleri belirle, problem sınırı dışındakileri ve soyut isimleri ele; kalanlar aday sınıflardır.

**CRC card** (CRC kartı) Class–Responsibility–Collaboration kartı: sınıfın adı, sorumlulukları ve çağırdığı (birlikte çalıştığı) sınıflar.

**God class** (Tanrı sınıf) Çok fazla bilgiye maruz kalan ve çok fazla kontrole sahip sınıf (bir anti-pattern).

**Use-case realization** (Use case gerçekleştirimi) Use case'leri genişletme ve inceltme süreci; bir senaryonun gerçekleştirimi sequence ya da collaboration diyagramıyla gösterilir.

**Collaboration diagram** (İşbirliği diyagramı) Nesneleri, aralarındaki bağlantıları ve gönderilme sırasına göre numaralanmış mesajları gösterir.

**Sequence diagram** (Sıra diyagramı) Nesneleri ve aralarındaki mesajları zaman sırasıyla (yukarıdan aşağıya) gösterir; çağrılar düz, dönüşler kesikli oktur.

# 8 UML: Sınıf İlişkileri ve Diyagramlar

**UML** Yazılımı belirlemek, görselleştirmek, inşa etmek ve belgelemek için standart (OMG) grafik modelleme dili; bir süreç değildir ve programlama dilinden bağımsızdır.

**Association** (İlişki) İki sınıfın örnekleri arasında, bir sınıfın işini yapabilmek için diğerini bilmesi gereken ilişki (düz çizgi; tek yönlüyse ok).

**Aggregation** (Topluluk) Bir sınıfın bir topluluğa ait olduğu ilişki (topluluk–üye); boş elmas topluluk tarafında; tek sahiplik yoktur.

**Composition** (Bileşim) Yaşam döngüsü kontrolü olan güçlü aggregation: parçalar bütün olmadan var olamaz; dolu elmas bütün tarafında.

**Inheritance** (Kalıtım) "is-a" (bir ... dır) ilişkisi; üst sınıfa bakan boş üçgen.

**Realization** (Gerçekleştirme) Bir sınıfın bir arayüzü gerçekleştirmesi; arayüze bakan kesikli çizgi ve boş üçgen.

**Dependency** (Bağımlılık) "uses-a" (kullanır) ilişkisi; bir sınıftaki değişiklik diğerini etkileyebilir; kesikli açık uçlu ok.

**Multiplicity** (Çokluk) Bir sınıfın bir örneğiyle ilişkilendirilebilecek diğer sınıf örneklerinin sayısı (1, 0..1, *, 1..*, n..m).